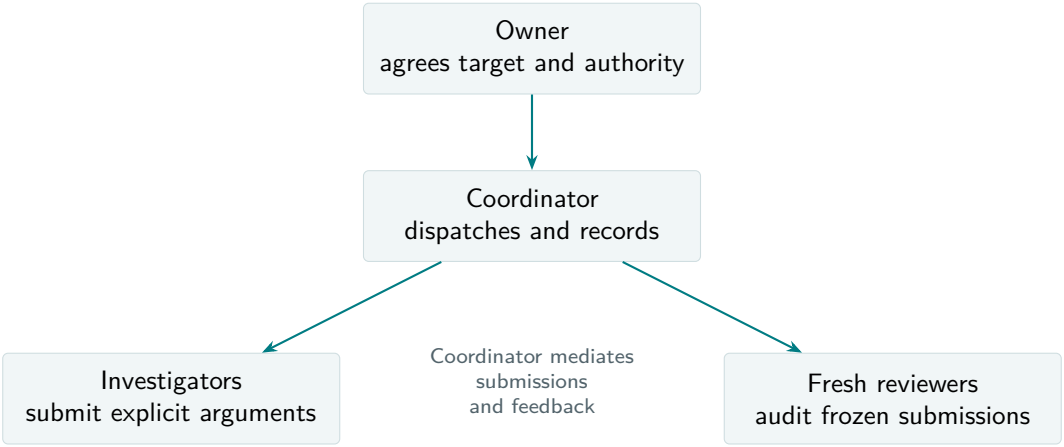
Setting up a multi-agent research experiment in Codex

From installation to an auditable research record

This guide explains how to organise a mathematical research experiment with a coordinator, independent investigators and fresh reviewers. It includes project files, configuration examples, launch prompts, submission limits and token-accounting rules.

The worked architecture comes from the reciprocal-simplex experiment in the `agentic-ai-math` repository. It is a reusable research protocol: replace the mathematical statement and approach assignments to study another problem.

What you will build



How to use this guide

Follow the steps in order. Prepare the files first, run a technical preflight, then explicitly launch research. The configuration examples use `gpt-6-astra` with `xhigh`, as in the worked experiment; availability must be checked in your own account.

Scope. This is a setup and documentation guide. Agent review provides a structured audit, not formal proof certification. Saved records should contain explicit proofs and concise reasoning summaries, not private chain-of-thought.

Software instructions were checked for the accompanying guide on 21 October 2026. Interfaces and configuration support can change; official documentation and the archived example are linked on the final page.

1–3 Install Codex and create the project

1. Download, install and sign in

Use the official desktop installation page. The documentation checked for this guide calls the installer the ChatGPT desktop app; select **Codex** inside it. Install the app and sign in with your ChatGPT account.

Open a **local project** so the coordinator can read and write the experiment files. An API key, Python and the command-line version of Codex are not required merely to coordinate the agents. Computation and PDF production may require additional tools. See the official quickstart for the current interface.

2. Create a clean folder

Create a folder such as `Documents/mathematical-research/experiment-01`. Open it in Codex and start a chat there. That chat is the coordinator. Keep previous results outside the new project when independence from earlier solutions matters.

You can create the files yourself or ask Codex to prepare them. Start with this instruction:

> Prepare the experiment files only. Do not launch research agents or investigate the mathematical problem yet.

3. Use a predictable directory structure

```
experiment-01/
  AGENTS.md
  START.md
  workflow.json
  .codex/
    config.toml
    agents/
      nonexistence_1.toml
      nonexistence_2.toml
      proof_1.toml                # also proof_2, proof_3, proof_4
      reviewer.toml
      literature.toml
  common/problem_statement.md
  instructions/
    coordinator.md
    nonexistence.md
    proof.md
    reviewer.md
    accounting.md
  approaches/                     # one assignment per approach
  starting_information/           # construction-only prior leads
  sources/                        # supplied papers and source index
  templates/                      # submissions, reviews, accounting
  runs/                           # all experiment outputs
```

Two kinds of file. `.codex/config.toml` and the agent TOML files configure Codex. `START.md`, `workflow.json` and the other research documents are conventions you introduce. The coordinator follows them because you instruct it to do so; the JSON file is not a built-in workflow engine.

4–5 Fix the target and the project rules

4. Write the exact mathematical problem

Save the initial statement as `common/problem_statement.md` and label it **S0**. Define the objects, assumptions, quantifiers and conclusion. Specify which parameters are prescribed and which may be chosen.

```
# Problem statement - S0

## Objects and notation
Define all objects and symbols.

## Assumptions and quantifiers
List every permitted assumption. State which parameters are fixed.

## Required conclusion
State the exact theorem or construction sought.

## Success criterion
A complete argument satisfying every requirement of S0.

## Partial results
Identify results that do not meet the full target: approximations,
extra hypotheses, restricted subclasses or numerical evidence.
```

For example, an exact identity for product densities must not quietly become an asymptotic statement or an identity for connected densities. A prescribed positive intensity must not become a freely chosen small intensity.

5. Put persistent rules in AGENTS.md

Codex reads `AGENTS.md` as project instructions. Personal or parent-directory instructions may also apply; inspect the effective rules before starting. See the official documentation.

```
# Research experiment rules

1. Do not launch research until the owner explicitly requests START.
2. The initial target is common/problem_statement.md, version S0.
3. Verify the agreed model and effort before research; no silent fallback.
4. Only the coordinator dispatches agents.
5. Use fresh contexts and explicit input packets for independent work.
6. Begin with two independent non-existence investigations.
7. Every frozen submission receives a fresh independent review.
8. A class-specific obstruction does not disprove the general target.
9. Each construction approach has four substantive submissions total.
10. After three unsuccessful reviewed submissions, an investigator may
    request a precise target amendment. Explicit owner approval is
    required. Attempt counts never reset automatically.
11. Save all outputs under runs/<run_id>/; preserve submitted versions.
12. Record exposed usage counters; unavailable means NR, not zero.
13. Save explicit proofs and concise reasoning and decision summaries.
14. Follow workflow.json and instructions/coordinator.md.
```

Decide revision authority in advance. State whether a reviewed general obstruction permits a coordinator-led restriction or whether every statement change requires owner approval. Record the logical necessity separately from the choice of repair. An obstruction assuming finite moments does not by itself restrict a target without that assumption.

6–7 Configure the model and an investigator

6. Create .codex/config.toml

The worked experiment used the following project configuration:

```
model = "gpt-6-astra"
model_reasoning_effort = "xhigh"

[agents]
enabled = true
max_concurrent_threads_per_session = 3
default_subagent_model = "gpt-6-astra"
default_subagent_reasoning_effort = "xhigh"
```

The concurrency value permits up to three spawned-agent threads in addition to the coordinator. It does not limit the number of agents used over the entire experiment. Schedule later investigators and fresh reviews as capacity becomes available.

Project configuration loads only for trusted projects. Use the normal trust flow for a folder you control. A saved model name neither proves account access nor necessarily switches a running coordinator. Start a fresh chat after setup and verify the actual settings. See configuration basics and the configuration reference.

7. Define specialised agent roles

Create one TOML file per role in `.codex/agents/`. The essential fields are `name`, `description` and `developer_instructions`. Here is `proof_1.toml`:

```
name = "proof_1"
description = "Develop an exact construction using approach 1."
model = "gpt-6-astra"
model_reasoning_effort = "xhigh"

developer_instructions = """
You are an independent construction investigator. Do not delegate.

Before mathematical work, require a coordinator packet containing:
- the active statement path, version and hash;
- the permitted inputs and your approach instructions;
- your submission number and assigned output paths.

Read the assigned project, proof and accounting instructions.
Do not read other construction tracks or previous experiments.
Prove every claimed property. Identify gaps and extra assumptions.
A partial result is not an exact-target success.

You have four substantive submissions total.
A target amendment does not reset this count.
Write only to assigned paths. Save an explicit proof and a concise
reasoning summary. Report unavailable usage fields as NR.
"""
```

Configuration does not launch research. Agent files define available roles. Explicit delegation requests or applicable project instructions cause the coordinator to use them. See the custom-agent documentation.

7 continued Define reviewers and the remaining roles

A reviewer receives a frozen submission and the exact target it claims to meet. Create `.codex/agents/reviewer.toml`:

```
name = "reviewer"
description = "Independently audit a frozen mathematical submission."
model = "gpt-6-astra"
model_reasoning_effort = "xhigh"

developer_instructions = """
Audit only the assigned frozen statement, submission and sources.
Do not delegate.

Check definitions, quantifiers, hidden assumptions, limiting
arguments, integrability and every claimed identity.

Distinguish a proved exact result, partial progress, an invalid
argument and an unresolved obligation. Explain your verdict.
Do not silently repair the proof or weaken the target.
Write the review to the assigned path.
"""
```

Role assignments

Use the same file pattern for the other roles. Put detailed mathematical assignments in the corresponding instruction and approach files.

Role	Responsibility
Coordinator	Main chat; dispatches, records decisions, updates statements and ledgers.
`nonexistence_1`	Investigates general obstructions without construction seeds.
`nonexistence_2`	Independently investigates obstructions from another assigned perspective.
`proof_1`–`proof_4`	Each pursues its own construction approach.
`reviewer`	Audits one frozen submission in a fresh context.
`literature`	Checks precise mathematical claims against primary sources.
Optional `coder`	Performs specific reproducible checks requested by the coordinator.

Fresh contexts are part of the design

A role definition can be reused, but each independent review should start in a fresh agent context. Do not copy the author's entire conversation or private work notes into the reviewer. Send the explicit proof and assigned evidence.

Similarly, the two obstruction investigators should not receive each other's arguments while developing their submissions. Construction agents may receive the same frozen starting-information packet after the obstruction phase. Label prior proofs as leads to verify, not accepted conclusions of the new experiment.

Optional skills. A reusable review procedure can be packaged as `.agents/skills/<name>/SKILL.md`. A skill defines a procedure; an agent file defines a role. Start with ordinary instruction files if that is simpler. See the skills documentation.

8 Write the workflow and coordinator instructions

Save the experiment's stages and limits in `workflow.json`:

```
{
  "initial_statement": "S0",
  "phases": [
    "preflight",
    "independent_obstructions",
    "obstruction_reviews",
    "target_decision",
    "independent_constructions",
    "submission_reviews",
    "final_synthesis"
  ],
  "construction_submission_limit": 4,
  "amendment_request_after_reviewed_failures": 3,
  "amendments_reset_attempt_count": false,
  "fresh_review_context": true,
  "only_coordinator_dispatches": true,
  "accounting_preflight_required": true
}
```

Explain its execution in `instructions/coordinator.md`:

1. Create one run directory. Freeze the intended inputs, initial statement and configuration with hashes.
2. Complete the model-access and accounting preflight before mathematical research.
3. Dispatch two independent obstruction investigators. Freeze each submission and commission a fresh review.
4. Decide the active target under the agreed revision authority. Record exactly what an obstruction proves and which repair has been chosen.
5. Give each construction investigator the approved target and permitted inputs. Keep ideas from separate tracks separate until synthesis.
6. Freeze each substantive submission, update its attempt count and obtain a fresh review. Send the investigator its own review for a permitted repair.
7. After three reviewed failures, allow an optional precise amendment request. Until the owner approves, retain the current target and attempt count.
8. After four failed submissions, mark that approach unsuccessful. Do not infer that the theorem is false.
9. Finish with an independent synthesis and a report of accepted results, failures, unresolved obligations and accounting coverage.

Keep decisions auditable

For every target change, save the old and new statements, the supporting reviews, a precise explanation, its authority and the owner's response when required. Broadcast each adopted version and its evidence to every role, including resumed agents.

Separate responsibilities. The coordinator owns shared targets and ledgers. Investigators own assigned submission files. Reviewers own assigned review files. An author must not overwrite a frozen submission after a reviewer has started reading it.

9–10 Control inputs and standardise submissions

9. Create an explicit dispatch packet

A packet states exactly what an agent is to read and produce. For example:

```
# Dispatch: proof_1, submission 1 of 4

Active statement: statements/S1.md
Statement SHA-256: <insert actual hash>

Permitted inputs:
- inputs/AGENTS.md
- inputs/instructions/proof.md
- inputs/approaches/01.md
- inputs/starting_information/<assigned files>

Assigned outputs:
- tracks/proof_1/submissions/v01.md
- tracks/proof_1/summaries/v01.md

Do not read other construction tracks, earlier experiments,
unassigned chats or private notes.

Required result:
An explicit construction satisfying S1, or a precise account of
which obligations remain unresolved.
```

Replace placeholders before dispatch. For the initial non-existence agents, omit construction seeds. References should resolve to the frozen run inputs rather than mutable originals.

These read restrictions establish **procedural independence**. They are not operating-system isolation. Agents sharing a workspace may technically be able to read other files. If strict isolation is required, use separate environments with enforced access restrictions.

10. Use submission and review templates

```
# Submission metadata
Role, target version/hash, submission number and timestamps.

# Exact claim
The precise result claimed.

# Construction or obstruction
Definitions and a complete explicit argument.

# Verification
Check each target requirement separately.

# Unresolved obligations
List gaps and extra assumptions.

# Reasoning summary and sources
Summarise the approach and decisive checks; identify source results.
```

A review should identify the statement and submission hashes, state its verdict, and justify it through mathematical checks. Include negative findings and unresolved obligations. “Looks correct” is not an adequate review. Numerical evidence belongs in a separate section and is not a proof of an exact identity.

11 Plan token, cost and runtime accounting

Put the accounting rules in `instructions/accounting.md`. Check what the runtime actually exposes before the research starts. A percentage of subscription usage is not a per-agent token count.

Record	Purpose
Run, agent and turn IDs	Identify whose work is being measured.
Role, target and attempt	Connect usage to the experiment.
Requested/observed model	Separate configuration from execution evidence.
Start and end timestamps	Establish runtime boundaries in UTC.
Input, cached input, output	Preserve exposed numeric counters and semantics.
Reasoning tokens	Record separately when exposed; check overlap with output.
Source and coverage	Identify exact, partial or unavailable measurements.
Cost assumptions	Record prices, model, tier, date and relevant context band.

Rules that prevent misleading totals

- Unknown measurements are `NR` in tables and `null` in JSON, never zero.
- For cumulative counters, use valid differences between checkpoints. Do not sum snapshots.
- Establish whether coordinator totals include child-agent usage before aggregating them.
- Do not add cached input again if it is already included in input tokens. Do not add reasoning tokens again if they are included in output.
- Record the observed service setting, including Fast where exposed. Do not infer it from response speed.
- Keep setup, technical preflight, research and later documentation usage distinct.
- Read only authorised session metadata. Do not collect private reasoning, credentials or unrelated chats to reconstruct usage.

Cost is a separate calculation

If I is total input, I_c its cached subset and O output, and the applicable prices per million tokens are known, a simple API-equivalent estimate is

$$C_{\text{estimate}} = \frac{(I - I_c)p_{\text{input}} + I_c p_{\text{cached}} + O p_{\text{output}}}{10^6}.$$

Use this only when the counter definitions and pricing categories support it. Account separately for other applicable fees or categories. Freeze the official price source and retrieval date. A subscription session's estimated API-equivalent cost is not its actual charge.

Runtime also needs a definition

Record per-agent elapsed time and the overall run's start and end. Summed agent runtime can exceed elapsed experiment time because agents run in parallel. Label both quantities.

Accounting gate. If exact per-agent telemetry is unavailable, tell the owner before research and obtain a decision on proceeding with `NR` fields or labelled estimates. Do not silently switch to a paid API workflow to obtain counters.

12 Validate the setup before doing research

Start a fresh coordinator chat after creating the configuration and role files. Run a technical check to make sure the intended roles, inputs, model settings and accounting mechanism work.

Copyable preflight prompt

```
Perform setup validation only. Do not investigate the mathematical
problem.

Read AGENTS.md, workflow.json and the coordinator and accounting
instructions. Check that every referenced file exists and that the
custom roles are available. Verify the actual coordinator model
and reasoning effort using available runtime evidence.

Create a prepared run directory and freeze the intended inputs.
Dispatch nonexistence_1 and nonexistence_2 in fresh contexts for a
technical check only. They must confirm their assigned roles,
permitted inputs, output paths and available model and usage
evidence. They must do no mathematical research.

Save accounting_capability.json. Distinguish verified fields from
unavailable fields. Report any missing capability before research
starts. These checks consume tokens but do not count as mathematical
submissions. Stop after preflight.
```

What a successful preflight establishes

1. All referenced instructions, approach files and source documents exist.
2. The coordinator can use the intended custom roles.
3. The requested model and effort have execution evidence, or the absence of verification is explicitly reported.
4. The accounting report identifies available fields, counter semantics and unresolved aggregation questions.
5. Each agent can write to its assigned output location.
6. The prepared run has a stable identifier and frozen inputs.

If a check fails, repair the setup and repeat the affected technical check. Do not label technical retries as mathematical submissions. Keep their usage in the preparation or preflight records.

A useful run directory

```
runs/<run_id>/
  inputs/                          # frozen setup and source additions
  statements/                      # S0, S1, ...; never overwrite
  packets/                         # input/output assignments
  tracks/<role>/submissions/       # v01, v02, v03, v04
  reviews/<role>_v01/              # fresh review of one version
  attempt_ledger.json
  decisions.md
  accounting_capability.json
  agent_accounting.jsonl
  hashes.json
  final_report.md
```

13–14 Launch, monitor and close the experiment

13. Start only after the preflight decision

Write the launch instructions into `START.md`. State the target, workflow, accounting gate, stopping rules and any owner decisions. Then explicitly launch with:

```
Start the research experiment by reading START.md. Continue in the
prepared run directory and follow AGENTS.md, workflow.json and the
coordinator instructions.

Begin with frozen S0 and two independent non-existence agents.
Obtain fresh reviews before adopting an obstruction or restriction.

Allow four substantive submissions per construction approach.
After three unsuccessful reviewed submissions, an investigator may
request a precise owner-approved statement change. Do not adopt it
without approval or reset the attempt count.

Preserve frozen submissions, reviews, concise reasoning summaries,
decision records and per-agent usage records.
```

14. Monitor records, not just conversational updates

Ask for compact status tied to saved statements, reviews and ledgers. A fictional example is:

```
Active target: S1
proof_1: submission 2/4, under review
proof_2: ended at 1/4 after an accepted class obstruction
proof_3: submission 3/4, repair pending
proof_4: submission 2/4, exact-target claim under review
Owner decisions pending: none
Accounting coverage: <measured coverage and missing fields>
```

Preserve failed submissions. A repair receives a new version and review; it does not replace the earlier files. A class-specific obstruction may end that class's track under the agreed rules, but it cannot establish non-existence for the general target.

Close with an independent synthesis

The final reviewer checks the exact accepted claim against the active statement and records remaining obligations. At this stage the coordinator may compare independent tracks and assemble the final report.

The deliverable should contain the frozen setup, all statement versions, dispatch packets, submissions, reviews, attempt ledger, decision history, source records and accounting data. Explain the architecture, chronology, accepted result, unsuccessful approaches and practical limitations.

What hashes establish. Hashes identify the exact file version that was submitted or reviewed. They do not establish mathematical correctness. The record should make a later human audit possible.

A shortcut for preparing the files

Once you understand the structure, you can ask Codex to create the scaffold instead of typing every file. Use this prompt in a new project, then inspect the generated files before preflight.

```
Prepare, but do not start, a multi-agent mathematical research
experiment in this folder.

Create AGENTS.md, START.md, workflow.json, .codex/config.toml,
.codex/agents/*.toml, common/problem_statement.md, the role and
accounting instructions, approach assignments and output templates.

The exact target is:
[PASTE THE FULL MATHEMATICAL STATEMENT HERE]

Use the following agreed model and reasoning effort:
[MODEL AND EFFORT]
Do not silently substitute if unavailable.

The coordinator is this chat. Only it dispatches agents.
Begin with two independent non-existence investigators. Then use
four independent construction approaches with assigned methods.
Every frozen submission receives a fresh independent review.

Allow four substantive submissions per construction approach.
After three unsuccessful reviewed submissions, an investigator may
request a precise statement amendment. Only explicit owner approval
authorises that amendment. Never reset attempt counts automatically.

Write an explicit rule for the authority to revise the target after
a reviewed general obstruction. Keep class-specific failures and
obstructions with additional moment assumptions correctly scoped.

Use fresh contexts, explicit permitted-input packets and assigned
output paths. Do not share new construction ideas between tracks.
Store all run outputs in runs/<run_id>/. Preserve frozen versions
and hashes. Save explicit proofs and concise reasoning summaries.

Require an accounting preflight. Record measured usage per agent
and turn when exposed, NR otherwise. Separate estimates from
charges, and elapsed experiment time from summed agent runtime.

Check consistency and list any missing decisions or files.
Do not dispatch agents or investigate the mathematics yet.
```

Before pressing START

- Replace every placeholder in the target, model settings and approach assignments.
- Check that the rules in `AGENTS.md`, the workflow and role files agree.
- Confirm that the initial investigators cannot accidentally encounter prior results in their assigned packets.
- Complete preflight and resolve model or accounting limitations.
- Confirm the statement-change authority and the four-submission ledger.

The preparation prompt is a convenience. The reviewed setup files and the preflight record are what define the experiment.

Documentation and worked example

Official product documentation

These links support the software setup. The research protocol, submission limits and independence rules are choices of this experiment.

1. Desktop app installation
   https://learn.chatgpt.com/docs/app
2. Quickstart
   https://learn.chatgpt.com/docs/quickstart
3. Configuration basics
   https://learn.chatgpt.com/docs/config-file/config-basic
4. Configuration reference
   https://learn.chatgpt.com/docs/config-file/config-reference
5. Subagents and custom agent definitions
   https://learn.chatgpt.com/docs/agent-configuration/subagents
6. Project instructions with AGENTS.md
   https://learn.chatgpt.com/docs/agent-configuration/agents-md
7. Reusable skills
   https://learn.chatgpt.com/docs/build-skills

The reciprocal-simplex experiment

The public experiment folder contains the research report, accounting and archived evidence. The frozen setup inputs provide the worked example for this guide.

Archive location	What to inspect
`.codex/`	Model configuration and custom agent definitions.
`AGENTS.md`	Persistent experimental rules.
`START.md`	Original launch instructions.
`workflow.json`	Stages, roles and submission policy.
`instructions/`	Coordinator, investigator, review and accounting procedures.
`common/`	The original exact mathematical target.
`approaches/`	Assigned construction methods.
`templates/`	Structured record formats.

Reusing the archive

Copy only the intended setup material into a new project. Include the hidden `.codex/` directory. Replace the mathematical target and approach assignments, check all paths, and provide your own required sources or skills. Personal skill bodies and copied third-party papers are omitted from the public archive.

Do not use the completed results as undisclosed input to a supposedly independent investigation. Keep the old experiment intact, create a new run, and repeat the technical preflight.

Prepared as a standalone guide from the documented experiment and the accompanying setup instructions. The editable LaTeX source is supplied with the PDF.